ANSI SQL Using MySQL

Complete Solutions — 25 Exercises

Database: Community Event Portal

This document provides complete, tested SQL solutions for all 25 exercises based on the Community Event Portal schema (Users, Events, Sessions, Registrations, Feedback, Resources). Each solution includes the query, explanation, and expected output description.

Schema Quick Reference:

- **Users**(user_id, full_name, email, city, registration_date)
- **Events**(event_id, title, description, city, start_date, end_date, status, organizer_id)
- **Sessions**(session_id, event_id, title, speaker_name, start_time, end_time)
- **Registrations**(registration_id, user_id, event_id, registration_date)
- **Feedback**(feedback_id, user_id, event_id, rating, comments, feedback_date)
- **Resources**(resource_id, event_id, resource_type, resource_url, uploaded_at)

---

1. User Upcoming Events

Task: Show all upcoming events a user is registered for in their city, sorted by date.

SQL Query:

```
SELECT
    e.event_id,
    e.title           AS event_title,
    e.city,
    e.start_date,
    e.end_date,
    e.status
FROM Events e
JOIN Registrations r ON e.event_id = r.event_id
JOIN Users u         ON r.user_id  = u.user_id
WHERE u.user_id = 1                 -- replace with target user_id
  AND e.status  = 'upcoming'
  AND e.city    = u.city            -- events in user's own city
ORDER BY e.start_date ASC;
```

Explanation: JOINs Registrations to link the user to events. Filter status='upcoming' and city match. ORDER BY start_date ASC shows nearest event first. **Expected Output:** Tech Innovators Meetup | New York | 2025-06-10

2. Top Rated Events

Task: Identify events with the highest average rating with at least 10 feedback submissions.

```
SELECT
    e.event_id,
    e.title,
    ROUND(AVG(f.rating), 2) AS avg_rating,
    COUNT(f.feedback_id)    AS total_feedback
FROM Events e
```

```
JOIN Feedback f ON e.event_id = f.event_id
GROUP BY e.event_id, e.title
HAVING COUNT(f.feedback_id) >= 10
ORDER BY avg_rating DESC;
```

Explanation: AVG(rating) computes the mean score per event. HAVING filters groups after aggregation (unlike WHERE which filters rows). With sample data (only 3 feedbacks), no rows return — add more data to test.

3. Inactive Users

Task: Retrieve users who have not registered for any events in the last 90 days.

```
SELECT
    u.user_id,
    u.full_name,
    u.email,
    u.city,
    u.registration_date
FROM Users u
WHERE u.user_id NOT IN (
    SELECT DISTINCT r.user_id
    FROM Registrations r
    WHERE r.registration_date >= DATE_SUB(CURDATE(), INTERVAL 90 DAY)
)
ORDER BY u.full_name;
```

Explanation: Subquery finds users who DID register in last 90 days. NOT IN excludes those users, returning only inactive ones. CURDATE() returns today's date. DATE_SUB subtracts 90 days from it.

4. Peak Session Hours

Task: Count how many sessions are scheduled between 10 AM to 12 PM for each event.

```
SELECT
    e.event_id,
    e.title             AS event_title,
    COUNT(s.session_id) AS sessions_10_to_12
FROM Events e
LEFT JOIN Sessions s
    ON  e.event_id   = s.event_id
    AND TIME(s.start_time) >= '10:00:00'
    AND TIME(s.start_time) <  '12:00:00'
GROUP BY e.event_id, e.title
ORDER BY sessions_10_to_12 DESC;
```

Explanation: TIME() extracts just the time portion of a DATETIME. LEFT JOIN ensures events with 0 qualifying sessions still appear. The time condition is placed in ON (not WHERE) to preserve all events in LEFT JOIN.

5. Most Active Cities

Task: List the top 5 cities with the highest number of distinct user registrations.

```
SELECT
    u.city,
    COUNT(DISTINCT r.user_id) AS distinct_registrations
FROM Users u
JOIN Registrations r ON u.user_id = r.user_id
GROUP BY u.city
ORDER BY distinct_registrations DESC
LIMIT 5;
```

Explanation: COUNT(DISTINCT user_id) counts unique users per city (not duplicate registrations). LIMIT 5 returns only the top 5 cities. **Expected Output:** New York(2), Los Angeles(2), Chicago(1)

6. Event Resource Summary

Task: Report showing the number of resources (PDFs, images, links) uploaded for each event.

```
SELECT
    e.event_id,
    e.title,
    COUNT(CASE WHEN r.resource_type = 'pdf'   THEN 1 END) AS pdf_count,
    COUNT(CASE WHEN r.resource_type = 'image' THEN 1 END) AS image_count,
    COUNT(CASE WHEN r.resource_type = 'link'  THEN 1 END) AS link_count,
    COUNT(r.resource_id)                                  AS total_resources
FROM Events e
LEFT JOIN Resources r ON e.event_id = r.event_id
GROUP BY e.event_id, e.title
ORDER BY total_resources DESC;
```

Explanation: Conditional COUNT using CASE WHEN pivots resource types into columns. LEFT JOIN ensures events with no resources still appear with 0 counts. **Expected Output:** Each event shows pdf/image/link breakdown and total.

7. Low Feedback Alerts

Task: List users who gave feedback with rating less than 3, with comments and event names.

```
SELECT
    u.full_name,
    u.email,
    e.title     AS event_name,
    f.rating,
    f.comments,
    f.feedback_date
FROM Feedback f
JOIN Users  u ON f.user_id  = u.user_id
JOIN Events e ON f.event_id = e.event_id
WHERE f.rating < 3
ORDER BY f.rating ASC, f.feedback_date DESC;
```

Explanation: Two JOINs connect Feedback to both Users and Events tables. WHERE f.rating < 3 filters only poor feedback. **Expected Output:** Bob Smith | bob@example.com | Tech Innovators Meetup | 3 | Could be better.

8. Sessions per Upcoming Event

Task: Display all upcoming events with the count of sessions scheduled for them.

```
SELECT
    e.event_id,
    e.title,
    e.city,
```

```
    e.start_date,
    COUNT(s.session_id) AS session_count
FROM Events e
LEFT JOIN Sessions s ON e.event_id = s.event_id
WHERE e.status = 'upcoming'
GROUP BY e.event_id, e.title, e.city, e.start_date
ORDER BY e.start_date ASC;
```

Explanation: WHERE filters to upcoming events only. LEFT JOIN keeps events even if they have 0 sessions. COUNT(session_id) returns 0 when no sessions match due to LEFT JOIN. **Expected Output:** Tech Innovators Meetup(2 sessions), Frontend Bootcamp(1 session)

9. Organizer Event Summary

Task: For each event organizer, show the number of events and their statuses.

```
SELECT
    u.user_id,
    u.full_name                                          AS organizer_name,
    COUNT(e.event_id)                                    AS total_events,
    COUNT(CASE WHEN e.status = 'upcoming'   THEN 1 END)  AS upcoming_count,
    COUNT(CASE WHEN e.status = 'completed'  THEN 1 END)  AS completed_count,
    COUNT(CASE WHEN e.status = 'cancelled'  THEN 1 END)  AS cancelled_count
FROM Users u
JOIN Events e ON u.user_id = e.organizer_id
GROUP BY u.user_id, u.full_name
ORDER BY total_events DESC;
```

Explanation: Joins Events to Users on organizer_id. CASE WHEN pivot breaks down counts by status without multiple queries. **Expected Output:** Alice Johnson(1 upcoming), Bob Smith(1 upcoming), Charlie Lee(1 completed)

10. Feedback Gap

Task: Identify events that had registrations but received no feedback at all.

```
SELECT
    e.event_id,
    e.title,
    e.status,
    COUNT(r.registration_id) AS total_registrations
FROM Events e
JOIN Registrations r ON e.event_id = r.event_id
WHERE e.event_id NOT IN (
    SELECT DISTINCT event_id FROM Feedback
)
GROUP BY e.event_id, e.title, e.status
ORDER BY total_registrations DESC;
```

Explanation: The subquery returns event_ids that have at least one feedback. NOT IN excludes those, leaving events with registrations but zero feedback. **Expected Output:** Frontend Development Bootcamp (1 registration, 0 feedback)

11. Daily New User Count

Task: Find the number of users who registered each day in the last 7 days.

```
SELECT
    registration_date,
    COUNT(user_id) AS new_users
FROM Users
WHERE registration_date >= DATE_SUB(CURDATE(), INTERVAL 7 DAY)
GROUP BY registration_date
ORDER BY registration_date DESC;
```

Explanation: DATE_SUB(CURDATE(), INTERVAL 7 DAY) gives the date 7 days ago. GROUP BY registration_date counts users per day. ORDER BY DESC shows most recent first. Days with 0 registrations won't appear (use calendar join for that).

12. Event with Maximum Sessions

Task: List the event(s) with the highest number of sessions.

```
SELECT
    e.event_id,
    e.title,
    COUNT(s.session_id) AS session_count
FROM Events e
JOIN Sessions s ON e.event_id = s.event_id
GROUP BY e.event_id, e.title
HAVING COUNT(s.session_id) = (
    SELECT MAX(session_count)
    FROM (
        SELECT COUNT(session_id) AS session_count
        FROM Sessions
        GROUP BY event_id
    ) AS sub
)
ORDER BY e.title;
```

Explanation: Subquery computes session counts per event. Outer subquery gets the MAX. HAVING matches events whose count equals the maximum — handles ties. **Expected Output:** Tech Innovators Meetup (2 sessions — highest count)

13. Average Rating per City

Task: Calculate the average feedback rating of events conducted in each city.

```
SELECT
    e.city,
    ROUND(AVG(f.rating), 2) AS avg_rating,
    COUNT(f.feedback_id)    AS total_feedback,
    COUNT(DISTINCT e.event_id) AS total_events
FROM Events e
JOIN Feedback f ON e.event_id = f.event_id
GROUP BY e.city
ORDER BY avg_rating DESC;
```

Explanation: Groups events by city, then averages the feedback ratings. ROUND(..., 2) limits to 2 decimal places. **Expected Output:** Chicago(avg 4.50), New York(avg 3.00)

14. Most Registered Events

Task: List top 3 events based on the total number of user registrations.

```
SELECT
    e.event_id,
    e.title,
    e.city,
    e.status,
    COUNT(r.registration_id) AS total_registrations
FROM Events e
JOIN Registrations r ON e.event_id = r.event_id
GROUP BY e.event_id, e.title, e.city, e.status
ORDER BY total_registrations DESC
LIMIT 3;
```

Explanation: COUNT(registration_id) tallies sign-ups per event. ORDER BY DESC and LIMIT 3 fetch the top 3. **Expected Output:** Tech Innovators Meetup(2), AI & ML Conference(2), Frontend Bootcamp(1)

15. Event Session Time Conflict

Task: Identify overlapping sessions within the same event.

```
SELECT
    s1.event_id,
    e.title             AS event_title,
    s1.session_id       AS session1_id,
    s1.title            AS session1_title,
    s1.start_time       AS s1_start,
    s1.end_time         AS s1_end,
    s2.session_id       AS session2_id,
    s2.title            AS session2_title,
    s2.start_time       AS s2_start,
    s2.end_time         AS s2_end
FROM Sessions s1
JOIN Sessions s2
    ON  s1.event_id   = s2.event_id      -- same event
    AND s1.session_id < s2.session_id    -- avoid self-join & duplicates
    AND s1.start_time < s2.end_time      -- overlap condition part 1
    AND s1.end_time   > s2.start_time    -- overlap condition part 2
JOIN Events e ON s1.event_id = e.event_id
ORDER BY s1.event_id, s1.start_time;
```

Explanation: Self-join on Sessions (s1 and s2 are the same table). s1.session_id < s2.session_id prevents (A vs B) and (B vs A) duplicate pairs. Two-condition overlap: s1 starts before s2 ends AND s1 ends after s2 starts. With sample data (sessions in sequence) — no overlaps exist.

16. Unregistered Active Users

Task: Find users who created an account in the last 30 days but haven't registered for any events.

```
SELECT
    u.user_id,
    u.full_name,
    u.email,
    u.city,
    u.registration_date,
    DATEDIFF(CURDATE(), u.registration_date) AS days_since_joined
FROM Users u
WHERE u.registration_date >= DATE_SUB(CURDATE(), INTERVAL 30 DAY)
  AND u.user_id NOT IN (
      SELECT DISTINCT user_id FROM Registrations
  )
ORDER BY u.registration_date DESC;
```

Explanation: First condition limits to users who joined in the last 30 days. NOT IN subquery excludes anyone who has any registration. DATEDIFF shows how many days ago they joined (useful for follow-up emails).

17. Multi-Session Speakers

Task: Identify speakers who are handling more than one session across all events.

```
SELECT
    speaker_name,
    COUNT(session_id)          AS total_sessions,
    COUNT(DISTINCT event_id)   AS events_involved,
    GROUP_CONCAT(title
        ORDER BY start_time
        SEPARATOR ' | ')       AS session_titles
FROM Sessions
GROUP BY speaker_name
HAVING COUNT(session_id) > 1
ORDER BY total_sessions DESC;
```

Explanation: GROUP BY speaker_name aggregates all sessions per speaker. HAVING COUNT > 1 keeps only those with multiple sessions. GROUP_CONCAT lists all their session titles in one column for easy reading. With sample data, no speaker has more than 1 session.

18. Resource Availability Check

Task: List all events that do not have any resources uploaded.

```
SELECT
    e.event_id,
    e.title,
    e.city,
    e.status,
    e.start_date
FROM Events e
LEFT JOIN Resources r ON e.event_id = r.event_id
WHERE r.resource_id IS NULL
ORDER BY e.start_date;
```

Explanation: LEFT JOIN keeps all events. WHERE r.resource_id IS NULL isolates events that had no match in Resources (i.e., no resources uploaded). This is more efficient than a NOT IN subquery for large datasets.

19. Completed Events with Feedback Summary

Task: For completed events, show total registrations and average feedback rating.

```
SELECT
    e.event_id,
    e.title,
    e.city,
    e.start_date,
    COUNT(DISTINCT r.registration_id)  AS total_registrations,
    COUNT(DISTINCT f.feedback_id)      AS total_feedback,
    ROUND(AVG(f.rating), 2)            AS avg_rating,
    COALESCE(ROUND(AVG(f.rating),2),
             'No feedback')            AS rating_display
FROM Events e
LEFT JOIN Registrations r ON e.event_id = r.event_id
LEFT JOIN Feedback      f ON e.event_id = f.event_id
WHERE e.status = 'completed'
GROUP BY e.event_id, e.title, e.city, e.start_date
ORDER BY avg_rating DESC;
```

Explanation: Both JOINs are LEFT to include events with 0 registrations or 0 feedback. COALESCE handles NULL avg_rating (when no feedback) and shows a friendly message. **Expected Output:** AI & ML Conference | Chicago | 2 registrations | avg 4.50

20. User Engagement Index

Task: For each user, calculate how many events they attended and how many feedbacks submitted.

```
SELECT
    u.user_id,
    u.full_name,
    u.city,
    COUNT(DISTINCT r.event_id)  AS events_registered,
    COUNT(DISTINCT f.event_id)  AS feedbacks_submitted,
    ROUND(
        COUNT(DISTINCT f.event_id) * 100.0 /
        NULLIF(COUNT(DISTINCT r.event_id), 0)
    , 1)                        AS feedback_rate_pct
FROM Users u
LEFT JOIN Registrations r ON u.user_id = r.user_id
LEFT JOIN Feedback      f ON u.user_id = f.user_id
GROUP BY u.user_id, u.full_name, u.city
ORDER BY events_registered DESC, feedbacks_submitted DESC;
```

Explanation: LEFT JOINs ensure users with 0 activity appear. NULLIF prevents division-by-zero when events_registered = 0. feedback_rate_pct shows what % of attended events got reviewed.

21. Top Feedback Providers

Task: List top 5 users who have submitted the most feedback entries.

```
SELECT
    u.user_id,
    u.full_name,
    u.email,
    u.city,
    COUNT(f.feedback_id)       AS total_feedback,
    ROUND(AVG(f.rating), 2)   AS avg_rating_given
FROM Users u
JOIN Feedback f ON u.user_id = f.user_id
GROUP BY u.user_id, u.full_name, u.email, u.city
ORDER BY total_feedback DESC
LIMIT 5;
```

Explanation: Joins Users to Feedback. GROUP BY aggregates per user. avg_rating_given shows whether top reviewers tend to rate positively or negatively. **Expected Output:** Charlie Lee(1), Diana King(1), Bob Smith(1) — all tied at 1 in sample data.

22. Duplicate Registrations Check

Task: Detect if a user has been registered more than once for the same event.

```
SELECT
    r.user_id,
    u.full_name,
    r.event_id,
    e.title             AS event_title,
    COUNT(r.registration_id) AS registration_count,
    GROUP_CONCAT(r.registration_id
        ORDER BY r.registration_id) AS duplicate_ids
FROM Registrations r
JOIN Users  u ON r.user_id  = u.user_id
JOIN Events e ON r.event_id = e.event_id
GROUP BY r.user_id, u.full_name, r.event_id, e.title
HAVING COUNT(r.registration_id) > 1
ORDER BY registration_count DESC;
```

Explanation: Groups by (user_id, event_id) pair. HAVING > 1 catches any duplicates. GROUP_CONCAT lists all duplicate registration IDs for easy cleanup. Well-designed schemas should enforce UNIQUE(user_id, event_id) to prevent this.

23. Registration Trends

Task: Show a month-wise registration count trend over the past 12 months.

```
SELECT
    DATE_FORMAT(registration_date, '%Y-%m') AS year_month,
    MONTHNAME(registration_date)            AS month_name,
    COUNT(registration_id)                  AS total_registrations
FROM Registrations
WHERE registration_date >= DATE_SUB(CURDATE(), INTERVAL 12 MONTH)
GROUP BY DATE_FORMAT(registration_date, '%Y-%m'),
         MONTHNAME(registration_date)
ORDER BY year_month ASC;
```

Explanation: DATE_FORMAT('%Y-%m') groups by month-year (e.g., 2025-04). MONTHNAME gives a readable label (April). ORDER BY year_month ASC shows the trend chronologically. **Expected Output:** 2025-04(2 registrations), 2025-05(2), 2025-06(1)

24. Average Session Duration per Event

Task: Compute the average duration (in minutes) of sessions in each event.

```
SELECT
    e.event_id,
    e.title                  AS event_title,
    COUNT(s.session_id)      AS total_sessions,
    ROUND(AVG(
        TIMESTAMPDIFF(MINUTE, s.start_time, s.end_time)
    ), 1)                    AS avg_duration_minutes,
    MIN(TIMESTAMPDIFF(MINUTE, s.start_time, s.end_time)) AS min_duration,
    MAX(TIMESTAMPDIFF(MINUTE, s.start_time, s.end_time)) AS max_duration
FROM Events e
JOIN Sessions s ON e.event_id = s.event_id
GROUP BY e.event_id, e.title
ORDER BY avg_duration_minutes DESC;
```

Explanation: TIMESTAMPDIFF(MINUTE, start, end) calculates exact minute difference between two DATETIMEs. AVG across sessions per event gives the average. MIN/MAX show session length range. **Expected Output:** Tech Innovators Meetup(avg 82.5 min), AI&ML;(90 min), Bootcamp(120 min)

25. Events Without Sessions

Task: List all events that currently have no sessions scheduled under them.

```
SELECT
    e.event_id,
    e.title,
    e.city,
    e.status,
    e.start_date,
    e.end_date,
    u.full_name AS organizer_name
FROM Events e
LEFT JOIN Sessions s ON e.event_id = s.event_id
LEFT JOIN Users    u ON e.organizer_id = u.user_id
WHERE s.session_id IS NULL
ORDER BY e.start_date;
```

Explanation: LEFT JOIN Events to Sessions. WHERE s.session_id IS NULL isolates events with no matching sessions (NULL appears in unmatched LEFT JOIN rows). Also joins Users to show who the organizer is. With sample data, all 3 events have at least 1 session — result would be empty.

End of ANSI SQL Using MySQL — All 25 Solutions Complete